Lab 9: Cryptography Fundamentals with Python

Course: FSCT 8561 - Security Applications

Instructor: Dr. Maryam R. Aliabadi

Scenario

You have been hired as a Junior Security Engineer at CipherCorp. Your first task is to audit the company's legacy systems and migrate them to modern cryptographic standards before an upcoming security audit.

Learning Objectives

- Run and observe cryptographic algorithms in action.
- Understand hashing, symmetric encryption, key derivation, and secure encryption methods.
- Analyze why some cryptographic methods are secure or insecure.
- Perform a basic brute-force attack to understand the importance of salt and complexity.

Part A — Hash Functions (One-Way Encryption)

Goal: Learn to generate cryptographic hashes and understand the concept of one-way encryption. Hashing is used for integrity. If our database is stolen, we don't want a hacker to see "Password123". We want them to see a useless string.

Task A1 — Basic Hashing

In [3]:
```
import hashlib
password = input("Enter a password: ")
hash_object = hashlib.sha256(password.encode())
print("SHA-256 hash:", hash_object.hexdigest())
```

```
SHA-256 hash: b7a7ce3f3cd75c40b29d6f20f60714e52a19385b214144ec3df4a28288bb07f8
```

Observations:

- Run twice with the same password → same hash?
  - Same Hash → Yes, hashing is deterministic.
- Change one character → observe hash change.

- Total change → Yes, even a small change in input results in a completely different hash (avalanche effect).

Modification:

- Change hashing algorithm: `hashlib.sha256` → `hashlib.md5` or `hashlib.sha1`.

Questions:

1. What happens when you slightly change the input?
   - The hash output changes drastically, demonstrating the avalanche effect.
2. Compare MD5 vs SHA-256 output length.
   - MD5 produces a 128-bit hash (32 hexadecimal characters), while SHA-256 produces a 256-bit hash (64 hexadecimal characters).
3. Why is hashing considered "one-way"?
   - Because it is computationally infeasible to reverse the hash back to the original input, making it a one-way function.

In [6]:
```
import hashlib
password = input("Enter a password: ")
hash_object = hashlib.md5(password.encode())
print("MD5 hash:", hash_object.hexdigest())
```

```
MD5 hash: 3c5e4920275101bb987ec2d15b284fb1
```

Task A2 — Hashing with Salt

Salt ensures that two users with the same password (e.g., "Spring2026") do not end up with the same hash in the database, protecting against Rainbow Table attacks.

In [7]:
```
import uuid

def hash_password(password):
    salt = uuid.uuid4().hex
    hashed = hashlib.sha256(salt.encode() + password.encode()).hexdigest()
    return hashed + ":" + salt

def check_password(stored_password, user_input):
    hashed, salt = stored_password.split(":")
    return hashed == hashlib.sha256(salt.encode() + user_input.encode()).hexdigest(

password = input("Enter password: ")
stored = hash_password(password)
print("Stored:", stored)
verify = input("Re-enter password: ")
print("Match:", check_password(stored, verify))
```

```
Stored: cacf042898d2bf4226bccf34f75a75f6f782aea26d05ce00f2e4c04fd36a619e:f0293f10bbe
8433c9441434e63b9ec46
Match: True
```

Questions:

1. What is the role of salt?
    - Salt adds random data to the input of a hash function, ensuring that identical inputs produce different hashes, this enhancing security against precomputed attacks.
2. Why does the hash change every time now?
    - Because a new random salt is generated each time, resulting in a different hash output even for the same password.

Part B — Symmetric Encryption with DES

Goal: Understand DES encryption, input requirements, and limitations. DES is the "antique" of the crypto world. Its 56-bit key size is no longer secure against modern computing power. We use it here to understand the mechanics of block ciphers.

In [16]:
```
from Crypto.Cipher import DES

key = "mycipher".encode("utf8")  # 8 bytes
message = "messags ".encode("utf8")  # 8 bytes

cipher = DES.new(key, DES.MODE_ECB)
cipher_text = cipher.encrypt(message)
print("Encrypted:", cipher_text)

cipher = DES.new(key, DES.MODE_ECB)
plain_text = cipher.decrypt(cipher_text)
print("Decrypted:", plain_text.decode())
```

```
Encrypted: b'\x7f\x85\x97\x90 \xd0\xd1\x0c'
Decrypted: messags
```

Observations:

- Run twice → compare outputs
    - Same output → Yes, DES is deterministic with the same key and message.
- Change message slightly
    - Total change → Yes, even a small change in the message results in a completely different ciphertext (avalanche effect).

Modification:

- Remove padding → observe error
- Fix padding with `.ljust(8)`
- Change key → observe output

Questions:

1. Why must DES input be exactly 8 bytes?
   - DES operates on 64-bit blocks (8 bytes), so the input must be exactly 8 bytes to fit the block size. If the input is shorter, it needs to be padded; if it's longer, it must be split into multiple blocks.
2. What happens if same key + same message is reused?
   - Reusing the same key and message would result in the same ciphertext, which could be exploited by attackers to identify patterns or perform statistical analysis.
3. Why is this a potential security issue?
   - It makes the encryption vulnerable to various attacks, such as frequency analysis or pattern recognition, especially if the same key is used for multiple messages.

Part C — AES Encryption

Goal: Understand AES encryption, key size, IV usage, and compare to DES. AES is the industry standard used by governments. Unlike ECB mode used above, we use CBC (Cipher Block Chaining) with an Initialization Vector (IV) to ensure that the same message encrypted twice looks different every time.

In [22]:
```
from Crypto.Cipher import AES

# AES-128 requires a 16-byte key
key = "secret-key-12345".encode("utf8")
iv = "This is an IV456".encode("utf8")

cipher = AES.new(key, AES.MODE_CBC, iv)
# Data must be a multiple of 16 bytes
message = "This is a secret message for lab".encode("utf8")

cipher_text = cipher.encrypt(message)
print(f"AES Encrypted: {cipher_text.hex()}")

# Decrypt using the same IV
decipher = AES.new(key, AES.MODE_CBC, iv)
plain_text = decipher.decrypt(cipher_text)
print(f"Decrypted: {plain_text.decode().strip()}")
```

```
AES Encrypted: 2b79e8faeb99a72d4fe658e38083789c80e7aceec1cb1824a1876e0f24863ee4
Decrypted: This is a secret message for lab
```

Observations:

- Run twice → same output?
  - Same Output
- Change IV → observe effect
  - Different output → Yes, changing the IV results in a different ciphertext even with the same key and message.

Modification:

- Use invalid key length → observe error
- Change message length → fix using `.ljust()`

Questions:

1. Why does AES require a specific key size?
   - AES supports key sizes of 128, 192, or 256 bits (16, 24, or 32 bytes) to provide different levels of security. Using an invalid key size will result in an error because the algorithm cannot operate with unsupported key lengths.
2. What is the purpose of IV?
   - The IV is used to ensure that the same plaintext encrypted with the same key produces different ciphertexts. This adds an extra layer of security by making the encryption process non-deterministic.
3. How is AES stronger than DES?
   - AES is stronger than DES because it uses a larger key size (128, 192, or 256 bits) compared to DES (56 bits), making it more resistant to brute-force attacks. Additionally, AES has a more complex structure and better diffusion properties, enhancing its overall security.

Part D — Key Derivation (PBKDF2)

Goal: Learn secure key generation from passwords using PBKDF2. Humans pick bad passwords. PBKDF2 makes it "expensive" for a hacker to guess passwords by running the hashing process thousands of times (iterations), slowing down brute-force attacks.

In [48]:
```
from Crypto.Protocol.KDF import PBKDF2
from Crypto import Random

password = "mypassword"
salt = Random.new().read(16)

key = PBKDF2(password, salt = b'fixedsalt123456', dkLen=16, count=10000)
print("Key:", key)
```

```
Key: b'3<\x1b\x85\xe3f,G\x88\xd8\xef\xa4jqm\x9a'
```

Observations:

- Run multiple times → compare outputs
  - Different outputs → Yes, because a new random salt is generated each time, resulting in a different derived key even for the same password.

Modification:

- Fix salt manually: `salt = b'fixedsalt123456'`
  - Same output → Yes, using a fixed salt results in the same derived key for the same password.
- Change iteration count → `count = 100` to `100000`
  - Different output → Yes, increasing the iteration count makes the key derivation process more computationally expensive, resulting in a different derived key.

Questions:

1. Why does changing salt change the key?
   - Changing the salt changes the input to the key derivation function, resulting in a different derived key even if the password remains the same. This is crucial for security, as it prevents attackers from using precomputed tables to reverse-engineer passwords.
2. What is the role of iterations?
   - Iterations increase the computational cost of deriving the key, making brute-force attacks more time-consuming and difficult for attackers.

Part E — Fernet (Simplified Secure Encryption)

Goal: Encrypt/decrypt safely using Fernet and observe randomness. In modern development, we often use Fernet because it handles the IV, padding, and integrity checks automatically, preventing "rookie" mistakes in implementation.

In [62]:
```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

message = "Secret message".encode()
encrypted = cipher.encrypt(message)
print("Encrypted:", encrypted)

decrypted = cipher.decrypt(encrypted)
print("Decrypted:", decrypted.decode())
```

```
Encrypted: b'gAAAAABp0_QK1cZB2S2clKV3yYLu0H8WxZMrxwgWCniPxZbgKt-uv42S3QwfesMfIVejhgM
ZYtDoEU_DybnXIr7_0SEYyq_SEw=='
Decrypted: Secret message
```

Observations:

- Run multiple times → compare outputs
  - the first part is remain the same (b'gAAAAABp0_), but the second part is different → Yes, Fernet generates a new random IV for each encryption, resulting in different ciphertexts even for the same plaintext and key.

- Modify key or save key to file → observe effect

Questions:

1. Why is Fernet safer than manual AES?
    - Fernet abstracts away the complexities of encryption, automatically handling IV generation, padding, and integrity checks. This reduces the risk of implementation errors that can lead to vulnerabilities.
2. What happens if the key is incorrect?
    - If the key is incorrect, Fernet will raise an error during decryption, indicating that the ciphertext cannot be decrypted with the provided key. This ensures that unauthorized access is prevented.

Part F — Broken Vault Challenge Challenge

Goal: Observe weaknesses in deterministic encryption and weak password hashing. A former employee was fired for poor security practices. They left behind an encrypted "vault" and a recovery PIN that was protected with a very weak hashing method. Your mission is to recover the PIN and identify why their encryption method was a failure.

Task F1: Brute-Forcing the Admin PIN

The employee used MD5 to hash a 4-digit PIN (e.g., "0000" to "9999") without using a salt. You have recovered the target hash: 81dc9bdb52d04dc20036dbd8313ed055.

Instructions: Complete the Python script below to "brute-force" the PIN. You must iterate through every possible 4-digit combination until you find the one that matches the target hash.

In [64]:
```
import hashlib

target_hash = "81dc9bdb52d04dc20036dbd8313ed055"

print("Starting Brute Force Attack...")

# --- STUDENT TODO: COMPLETE THE LOGIC BELOW ---

# 1. Create a loop that goes from 0 to 9999
# 2. Use str(i).zfill(4) to ensure numbers like 7 become "0007"
# 3. Hash the string using hashlib.md5()
# 4. Compare the result to the target_hash
# 5. Print the PIN if you find a match

# YOUR CODE HERE:
for i in range(10000):
    pin = str(i).zfill(4)
```

```
    hash_object = hashlib.md5(pin.encode())
    if hash_object.hexdigest() == target_hash:
        print(f"PIN found: {pin}")
        break

# --- END TODO ---
```

```
Starting Brute Force Attack...
PIN found: 1234
```

Task F2: Analyzing the ECB Leak

The employee used DES in ECB mode to encrypt two different messages using the same key. Look at the output of the code below and answer the question that follows.

In [65]:
```
from Crypto.Cipher import DES

key = "security".encode("utf8")
cipher = DES.new(key, DES.MODE_ECB)

# Encrypting two separate data packets
packet_1 = "DATA_REQ".encode("utf8")
packet_2 = "DATA_REQ".encode("utf8")

print(f"Packet 1 Ciphertext: {cipher.encrypt(packet_1).hex()}")
print(f"Packet 2 Ciphertext: {cipher.encrypt(packet_2).hex()}")
```

```
Packet 1 Ciphertext: 6e9501ebbf08ec18
Packet 2 Ciphertext: 6e9501ebbf08ec18
```

Question: If an attacker intercepts these two packets and sees that the encrypted "Ciphertext" is identical, what information have they gained even if they don't have the key? Why is this a major security vulnerability for a database containing many identical values (like "Male/Female" or "Yes/No")?

- The attacker can infer that the plaintexts for both packets are identical, which can lead to information leakage. In a database with many identical values, this vulnerability allows attackers to identify patterns and potentially reverse-engineer the data, compromising the confidentiality of the information stored.

Deliverables

A PDF containing:

- Screenshots of code outputs for each part (A–F)
- Your completed code for the Brute-Force loop.
- Answers to all questions and observations
- Submit as PDF: `Lab9-FirstName-LastName-StudentID.pdf`

Good Luck!

In [ ]: